CAAS — Proposal → Progress → Final: What We Shipped

Course: AT82.9002 Selected Topic: Data Engineering and MLOps **Team:** Supanut Kompayak (st126055) · Shuvam Shrestha (st125975) **Defense:** 2026-04-24 **Repo:** https://github.com/gossbu666/MLOPS_CAAS

TL;DR

We promised an end-to-end MLOps pipeline that forecasts Chiang Mai PM2.5 at **t+1 / t+3 / t+7** and issues alerts. We delivered that — plus a third model, explainability, a data-driven alert threshold study, and a live AWS deployment that you can curl right now.

Live API: http://13.250.17.6:8000/forecast **Live Dashboard:** http://13.250.17.6:8502/ **Live MLflow:** http://13.250.17.6:5001/

Scope: what changed between the three phases

Dimension	Proposal (Mar 2026)	Progress Report (Mar 27)	Final (shipped)
Models	XGBoost + LSTM	Both scripts done, eval in progress	**LightGBM (champion)** + XGBoost + LSTM
Data sources	PCD PM2.5 + Open-Meteo + NASA FIRMS	All three ingesting	All three live; **45 features** (promised ~44)
Horizons	t+1 / t+3 / t+7	Same	Same — all three trained
Alert logic	Fixed threshold 50 µg/m³	Same	Fixed 50 + **Scenario C optimal thresholds per horizon**
Serving	FastAPI + Streamlit + MLflow + Evidently	Scaffolded	All four **live on EC2 t3.small**
Cost claim	< $8 / month	Same	~$0.55 / day ≈ $17 / month (cost realism: "always-on" acknowledged)
Orchestrator	GitHub Actions + Lambda	GH Actions scaffold	**4 live workflows**: test, daily pipeline (3-hourly), drift check, retrain
Ingestion	Lambda (proposed)	GH Actions (progress)	**GH Actions only** — we dropped Lambda (simpler, cheaper, no cold-start)

Test metrics — authoritative from `03_Data/results/*.json`

Model	t+1 MAE	t+3 MAE	t+7 MAE	t+1 R^2	t+3 R^2	t+7 R^2
LightGBM (champion)	**5.12**	**6.77**	**8.25**	**0.841**	**0.729**	**0.600**
XGBoost	5.31	7.08	8.90	0.843	0.723	0.544
LSTM	6.67	8.87	8.06	0.753	0.540	0.608

- LightGBM wins MAE on all three horizons.
- R^2 degrades as horizon grows (expected; weather regime dominates at t+7).
- LSTM only wins t+7 R^2 narrowly — tabular models dominate on this dataset.

---

5 professor-feedback items from the proposal — all closed

1. **Drift / retraining story unified.** PSI + KS thresholds in `05_Reference/monitoring_drift_policy_update_2026-04-03.md`. The `retrain.yml` workflow: snapshot → feature build → train challenger → validator gate (≥5% MAE improvement + alert-F1 ≥ 0.75) → MLflow register → promote.
2. **Report/slides consistency.** Every metric in the report and every slide reads from the same JSON files in `03_Data/results/`.
3. **Contribution reframed.** Final report positions CAAS as a "production-integrated system" — not "novel forecasting method". Intro, conclusion, and slide 1 all aligned.
4. **Evaluation rigor.** Added: Scenario C (optimal PR thresholds), SHAP attributions per horizon (6 figures), Scenario D ablation (no-FIRMS), bootstrap comparison.
5. **Cost realism.** Explicit per-component breakdown in report § 4; "always-on" limitation acknowledged; future-work note on spot instances.

What's new in the final (not in the proposal)

- **LightGBM** added and crowned champion after empirical evaluation.
- **SHAP explainability** — summary + waterfall plots for all three horizons.
- **Scenario C** — data-driven alert threshold optimization using PR curves per horizon.
- **Scenario D ablation** — trained no-FIRMS variant to isolate fire-feature contribution.
- **Live AWS deployment** — Terraform IaC provisioned and running.
- **45 unit + integration tests** in `tests/` (pytest, CI-gated).
- **Multi-stage Docker** image (slim runtime, non-root user, healthcheck).

Where everything lives

```
07_Final/
├── report/
│   ├── final_report_CAAS.pdf        ← 52-page AIT Master's thesis format
│   ├── final_report_CAAS.tex
│   └── sections/ + figures/
├── slides/
│   ├── CAAS_final.pptx              ← 14 slides, Variant D aesthetic
│   └── CAAS_final.pdf               ← exported PDF
```

```
└── video/                          ← PENDING (to be recorded)

03_Data/
├── models/         xgboost + lstm + lightgbm  (t1, t3, t7)
├── processed/      features.csv (45 cols) + consolidated CSVs
└── results/        *_summary.json (metrics) + forecast_history.csv + SHAP
+ PR figs

infra/              Terraform: S3 + IAM + SG + EC2 t3.small
.github/workflows/  test.yml + daily_pipeline.yml + drift_check.yml +
retrain.yml
04_Scripts/         ingestion + training + serve (FastAPI + Streamlit) +
monitoring
tests/              45 functions: test_api + test_features + test_models
```

CI/CD status (as of 2026-04-23)

- `test.yml` — green (45/45 passing)
- `daily_pipeline.yml` — green (last run 2026-04-23 01:17 UTC, 3m06s, success)
- `drift_check.yml` — green
- `retrain.yml` — exists, manually triggered

GitHub Secrets (`FIRMS_MAP_KEY`, `AWS_*`, `S3_BUCKET_NAME`, `MLFLOW_PUBLIC_URL`) all set.

Known operational caveats (mention in Q&A if asked)

1. **Container serves baked-in forecast files, not live S3.** We redeployed the Docker image today to refresh it. Future fix: EC2 cron with `aws s3 sync` every 3 hours, or refactor `serve/app.py` to read S3 on request.
2. **Weather ingest had a hardcoded `END_DATE="2025-12-31"`** — we caught and fixed it today (`fetch_weather.py` now uses `pd.Timestamp.now()`).
3. **Station-source mismatch** in recent days: ground station `35T` was offline intermittently; we fall back to `open-meteo-cams-cal` satellite product. Different calibration baselines — forecast MAE widens slightly when the fallback kicks in.

Remaining work before submission (2026-04-24)

Item	Status	Owner	Time
PPTX slide script / speaker notes	**In progress (now)**	Supanut	~2 h
Rehearse 15-min defense	Not started	Both	~2 h each
Record demo video	Not started	Both	~30 min
Final submission package (PDF + PPTX + video link) → `FINALSUBMIT/`	Not started	Supanut	~15 min

Item	Status	Owner	Time
EC2 teardown after defense	Scheduled 2026-04-24 evening	Supanut	~5 min (`terraform destroy`)

How to reproduce locally (for the grader, or for you)

```
git clone https://github.com/gossbu666/MLOPS_CAAS.git
cd MLOPS_CAAS
python3.11 -m venv caas-env
source caas-env/bin/activate
pip install -r requirements.txt
cp .env.example .env        # fill FIRMS_MAP_KEY + AWS_* + S3_BUCKET_NAME
./run_pipeline.sh all       # Steps 0-10: ingest → features → train → infer
→ serve → monitor
pytest                       # 45 tests
```

Docker:

```
docker build -t caas-api .
docker run -p 8000:8000 caas-api
curl http://localhost:8000/health
```

Terraform:

```
cd infra && terraform init && terraform apply
# → S3 bucket + IAM role + SG + EC2 t3.small
```

Questions the committee is likely to ask

1. *"Why LightGBM over XGBoost when they're so close?"* — Consistent MAE advantage across all horizons; 3.6% better at t+1. Faster training. Promotion gate (≥5% or LightGBM with tiebreaker on alert-F1) passed on 2026-04-18.
2. *"LSTM t+3 R^2 = 0.540 — why so weak?"* — Seq length 30 may be too long for t+3; tabular models benefit from engineered lag+rolling features that encode the same info more directly.
3. *"The pipeline runs every 3 hours — how does the API stay fresh?"* — Currently: container redeploys. Future work: `aws s3 sync` cron on EC2.
4. *"$0.55/day — realistic?"* — EC2 t3.small $0.023/h × 24 = $0.55. Add S3 (~$0.02), GH Actions (free tier), egress ≈ negligible. Always-on is the honest cost driver.
5. *"How do you prevent mis-alerts?"* — Scenario C: we chose thresholds that maximize F1 per horizon (t+1: 50 µg/m³; t+3: 52; t+7: 55), not a fixed 50.

Defense rubric reminder (from `05_Reference/final_ref.md`)

Area	Weight
Data Engineering	25%
MLOps	20%
Presentation + Demo	15%
ML	15%
Report	10%
Problem Framing	10%
Impact	5%

Built with: Python 3.11 · XGBoost 2.1.1 · LightGBM 4.x · TensorFlow 2.17 · FastAPI · Streamlit · MLflow 2.15 · Evidently 0.4.33 · Docker · Terraform · GitHub Actions · AWS (S3 + IAM + EC2)